СОГЛАСОВАНО

Председатель муниципальной предметно-методической комиссии
по информатике

_____ /_____/
«___» _____ 2026 г.

УТВЕРЖДАЮ

Руководитель органа местного самоуправления
в сфере образования

_____ /_____/
«___» _____ 2026 г.

Муниципальный этап ВсОШ по информатике

Решения задач (9-11 классы)

Учебный год: **2026/27**

Задача A. Стабилизация таймлайна

Условие (кратко)

Даны `t1..tn`. Нужно найти минимальное `D`, чтобы можно было выбрать `a1 < a2 < ... < an` и для каждого `i` выполнялось `|ai - ti| <= D`.

Идея решения

Ищем ответ бинарным поиском по `D`.

Проверка `ok(D)` (жадная, линейная):
- поддерживаем `prev` — уже выбранное `a(i-1)`;
- для текущего `ti` допустимый интервал `[ti-D, ti+D]`;
- выбираем минимально допустимое `ai = max(prev+1, ti-D)`;
- если `ai > ti+D`, построить последовательность нельзя.

Если проверка истинна, пробуем меньшее `D`; иначе большее.

Сложность

- Проверка: `O(n)`
- Полное решение: `O(n log V)`, где `V` — диапазон значений.

Эталон (Python)

```
n = int(input().strip())
t = list(map(int, input().split()))

def ok(D: int) -> bool:
    prev = -10**30
    for x in t:
        left, right = x - D, x + D
        cur = max(prev + 1, left)
        if cur > right:
            return False
        prev = cur
    return True

lo, hi = 0, 2 * 10**9
while lo < hi:
```

```
    mid = (lo + hi) // 2
    if ok(mid):
        hi = mid
    else:
        lo = mid + 1

print(lo)
```

Задача B. Маршрут с двумя режимами

Условие (кратко)

У ребра две стоимости: `day` и `night`.
Старт в вершине `1` в режиме `day`. В любой вершине можно переключить режим за `p[v]`. Нужно минимально добраться до вершины `n`.

Идея решения

Дейкстра на графе состояний:
- состояние `(v,0)` — вершина `v`, режим `day`;
- состояние `(v,1)` — вершина `v`, режим `night`.

Переходы:
1. По ребру `(v,to)`:
   - из `(v,0)` в `(to,0)` с ценой `day`;
   - из `(v,1)` в `(to,1)` с ценой `night`.
2. Переключение режима:
   - `(v,0) -> (v,1)` за `p[v]`;
   - `(v,1) -> (v,0)` за `p[v]`.

Ответ: `min(dist[n][0], dist[n][1])`, если бесконечность — `-1`.

Сложность

`O((n+m) log n)`.

Задача C. Цепная склейка отчетов

Условие (кратко)

Даны строки `s1..sm`, порядок фиксирован. При склейке соседних строк берется максимально возможное перекрытие: суффикс `si` = префикс `s(i+1)`. Нужно минимизировать длину итоговой строки.

Идея решения

Итоговая длина:

`sum(len(si)) - sum(overlap(si, s(i+1)))`.

Остается быстро считать `overlap(A, B)` для каждой пары соседних строк.
Это можно делать через префикс-функцию:
- строим строку `B + '#' + suffix_of_A`,
- максимальное значение `pi` в конце дает длину совпавшего префикса `B`.

Практически проще:
- взять `T = B + '#' + A_tail`, где `A_tail` — последние `min(len(A), len(B))` символов `A`;
- посчитать `pi` на `T`, последнее значение и есть перекрытие.

Сложность

`O(суммарной длины всех строк)`.

Задача D. Квадраты датчиков

Условие (кратко)

По набору точек нужно посчитать число осевых квадратов (стороны параллельны осям), чьи 4 вершины есть в наборе.

Идея решения

1. Храним все точки в `set` для проверки наличия за `O(1)` в среднем.
2. Перебираем пары точек на одной горизонтали:
   `(x1, y)`, `(x2, y)`, `x1 < x2`.
3. Сторона квадрата `d = x2 - x1`.
4. Проверяем наличие верхних вершин:
   `(x1, y+d)` и `(x2, y+d)`.

Считаем только квадраты "вверх", чтобы каждый квадрат учитывался один раз.

Сложность

`O(n^2)` по числу точек.

Задача E. Сегментация шума

Условие (кратко)

Разбить массив `a1..an` на `k` непустых подряд идущих сегментов, минимизируя сумму цен сегментов, где
`cost(l,r) = max(a[l..r]) - min(a[l..r])`.

Идея решения

1. Предвычисляем `cost[l][r]` за `O(n^2)`.
   Для фиксированного `l` двигаем `r` вправо, поддерживая текущие `min/max`.
2. Динамика:
   `dp[j][i]` — минимум для первых `i` элементов при разбиении на `j` сегментов.

Переход:
`dp[j][i] = min(dp[j-1][t] + cost[t+1][i])`, где `j-1 <= t < i`.

База:
- `dp[1][i] = cost[1][i]`.

Сложность

- Предвычисление `cost`: `O(n^2)`.
- DP: `O(k*n^2)`.
- Память: `O(k*n)` (или `O(n)` при оптимизации по слоям).

Эталонный каркас (Python-псевдокод)

```
# cost[l][r] готово
INF = 10**30
dp = [[INF] * (n + 1) for _ in range(k + 1)]
dp[0][0] = 0

for j in range(1, k + 1):
    for i in range(1, n + 1):
        for t in range(j - 1, i):
            dp[j][i] = min(dp[j][i], dp[j - 1][t] + cost[t + 1][i])
```

```
print(dp[k][n])
```

Примечание для жюри

При публикации разборов после тура рекомендуется:
1. добавить эталонные реализации на C++17 и Python 3;
2. привести по 2-3 характерных теста на каждую задачу;
3. отдельно разобрать типовые ошибки участников.